Incident Response Runbook

Enterprise DevSecOps Security Lab

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Document Type	Falco Runtime Security Response Procedures
Environment	Private Ubuntu VM, Kind Kubernetes, Falco, Prometheus, Grafana
Classification	Portfolio / Private Lab Evidence

Document Intent
This professional portfolio artifact describes the design, implementation, validation, and operational usage of runtime detection, alerting, dashboards, or response artifacts created inside the private Enterprise DevSecOps lab.

Coverage Area	Included in this document
Runtime detection	Falco event logic, validation evidence, and triage workflow
Security observability	Prometheus metrics, Grafana dashboarding, alert health checks
Response readiness	Analyst actions, containment commands, retest criteria, evidence handling
Portfolio value	Professional documentation showing detection engineering, SOC process, and cloud-native security maturity

Lab boundary: no external systems were targeted. All attack simulations, detections, and response workflows were performed inside an isolated private VM and Kind Kubernetes environment.

Runbook Purpose

This runbook defines a repeatable incident response process for runtime security alerts generated by Falco and surfaced through Prometheus and Grafana. It is designed for the Enterprise DevSecOps lab but follows an enterprise operating model: detect, validate, contain, eradicate, recover, document, and improve.

Severity Model

Severity	Trigger Example	Response Time	Primary Action
SEV-1 Critical	Host filesystem access, kernel event drops sustained, confirmed credential exposure	Immediate	Contain workload, preserve evidence, notify owner
SEV-2 High	Privileged pod, sensitive file read, suspicious shell in app namespace	< 30 minutes	Isolate pod, review RBAC and image provenance
SEV-3 Medium	Single warning event from test namespace or known scanner	Same business day	Validate and document exception
SEV-4 Low	Expected lab event or known benign action	Next review cycle	Record and tune if noisy

Initial Triage Checklist

- Capture alert time, rule name, priority, pod, namespace, node, container image, command, user, and file path.
- Confirm whether the event came from a test namespace, CI job, expected scanner, or unauthorized workload.
- Check recent deployments, ArgoCD sync events, image digest changes, and service account bindings.
- Review whether Falco event drops or output queue drops occurred during the same window.
- Preserve logs and Kubernetes object descriptions before deleting or restarting affected resources.

Core Commands

```
# Identify Falco alert context
kubectl logs -n falco -l app.kubernetes.io/name=falco --since=30m | egrep -i
  "critical|warning|shadow|privileged"

# Collect pod and workload state
kubectl get pod -A -o wide | grep <pod-name>
kubectl describe pod <pod-name> -n <namespace>
kubectl get pod <pod-name> -n <namespace> -o yaml > evidence-pod.yaml

# Inspect service account and RBAC
kubectl get pod <pod-name> -n <namespace> -o jsonpath='{.spec.serviceAccountName}'
kubectl auth can-i get secrets --as=system:serviceaccount:<namespace>:<serviceaccount> -n
  <namespace>

# Contain an unsafe workload
kubectl scale deployment <deployment> -n <namespace> --replicas=0
kubectl delete pod <pod-name> -n <namespace>
```

Playbook A: Sensitive File Read

Phase	Action
Detect	Falco emits sensitive file read alert with file path, command, user, image, pod, and namespace.
Validate	Confirm file path, parent process, workload owner, and whether this is an approved scanner.
Contain	If unauthorized, scale down deployment or delete pod. Preserve YAML, logs, and image digest first.
Eradicate	Patch workload to remove hostPath, privileged access, or unnecessary file access.
Recover	Redeploy hardened image and verify no repeat alerts.
Lessons Learned	Add allowlist only for approved tools and require exception expiry dates.

Playbook B: Privileged Pod or HostPath Abuse

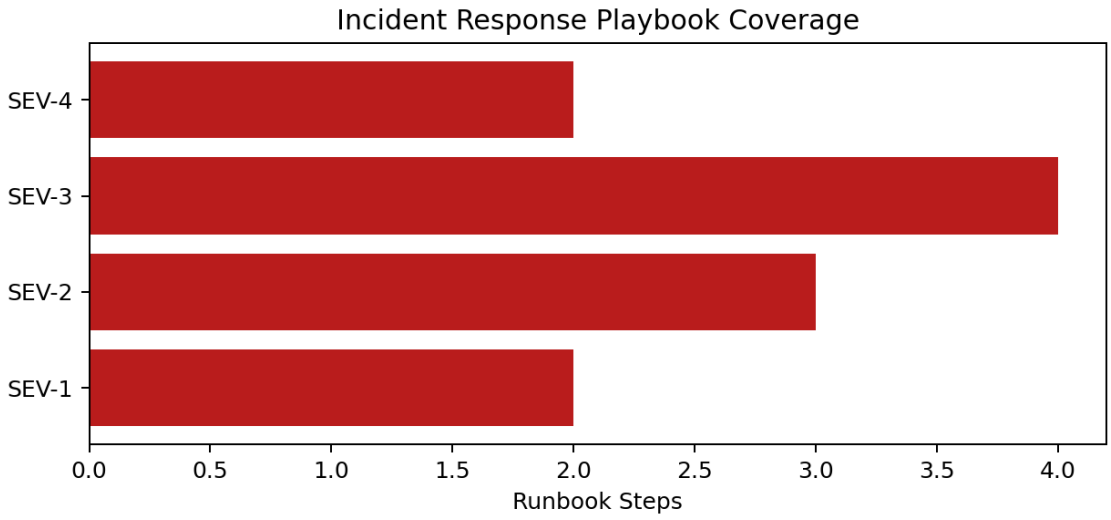
Phase	Action
Detect	Alert indicates privileged behavior, host filesystem access, or sensitive file reads from a host mount.
Validate	Run kubectl describe pod and inspect securityContext.privileged and hostPath volumes.
Contain	Delete the pod or scale the deployment to zero.
Prevent	Apply Pod Security restricted labels and Gatekeeper/Kyverno policies to block privileged pods.
Retest	Reapply the same unsafe pod manifest and confirm admission is denied.

Playbook C: Falco Detection Health Degraded

Dropped kernel events or output queue drops mean detection quality may be degraded. Treat this as a monitoring integrity incident because alerts could be missed.

```
# Query alert health metrics
sum(increase(falcosecurity_scap_n_drops_total[5m]))
sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m]))

# Validate resources
kubectl top pod -n falco
kubectl describe pod -n falco -l app.kubernetes.io/name=falco
kubectl logs -n falco -l app.kubernetes.io/name=falco --since=30m
```



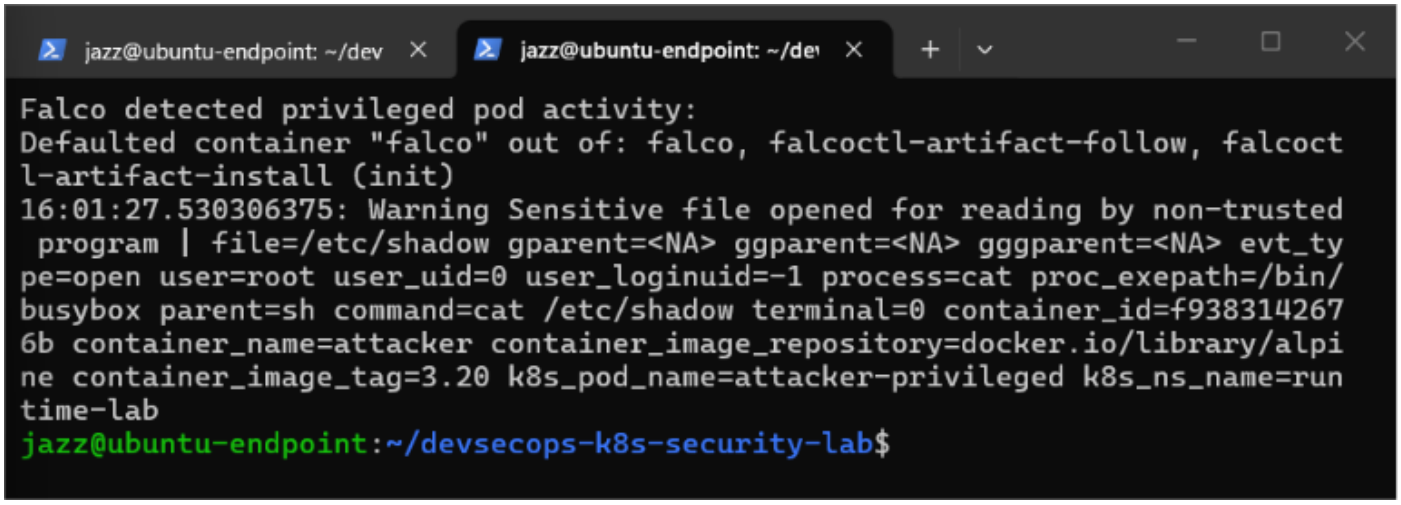
Incident response coverage by severity class.

Evidence Collection Requirements

Evidence Type	Command / Source	Purpose
Falco logs	kubectl logs -n falco ...	Prove detection event and context
Pod YAML	kubectl get pod -o yaml	Preserve security context and volume configuration
RBAC state	kubectl auth can-i / get rolebindings	Determine blast radius
Prometheus query	falcosecurity_falco_rules_matches_total	Show event count and timing
Grafana screenshot	Falco runtime security dashboard	Show operational visibility

Closure Criteria

- Root cause identified and documented.
- Unsafe pod, RBAC, or image condition removed or accepted via exception.
- Preventive control validated through a negative retest.
- Falco and Prometheus alert health confirmed with zero unexpected drops.
- Runbook, dashboard, or rule tuning updated if the incident exposed a detection gap.



Evidence example used by the runbook for sensitive file triage.

References

Falco rules basic elements - https://falco.org/docs/concepts/rules/basic-elements/

Falco supported fields for rule conditions and outputs - https://falco.org/docs/reference/rules/supported-fields/

Falco metrics - https://falco.org/docs/concepts/metrics/

Falco alert forwarding and JSON outputs - https://falco.org/docs/concepts/outputs/forwarding/

Prometheus alerting rules - https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

Grafana dashboard JSON model - https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/

Grafana dashboard import workflow - https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/import-dashboards/